



MSE 153: Introduction to IT

Introduction to Operating System Concepts



Lecture Outline

- What is an Operating System?
- Some Useful Terms
- OS Tasks
- Forms of Process Management
- Booting and System Initialization
- Administrator Account
- Moving around Linux and DOS.

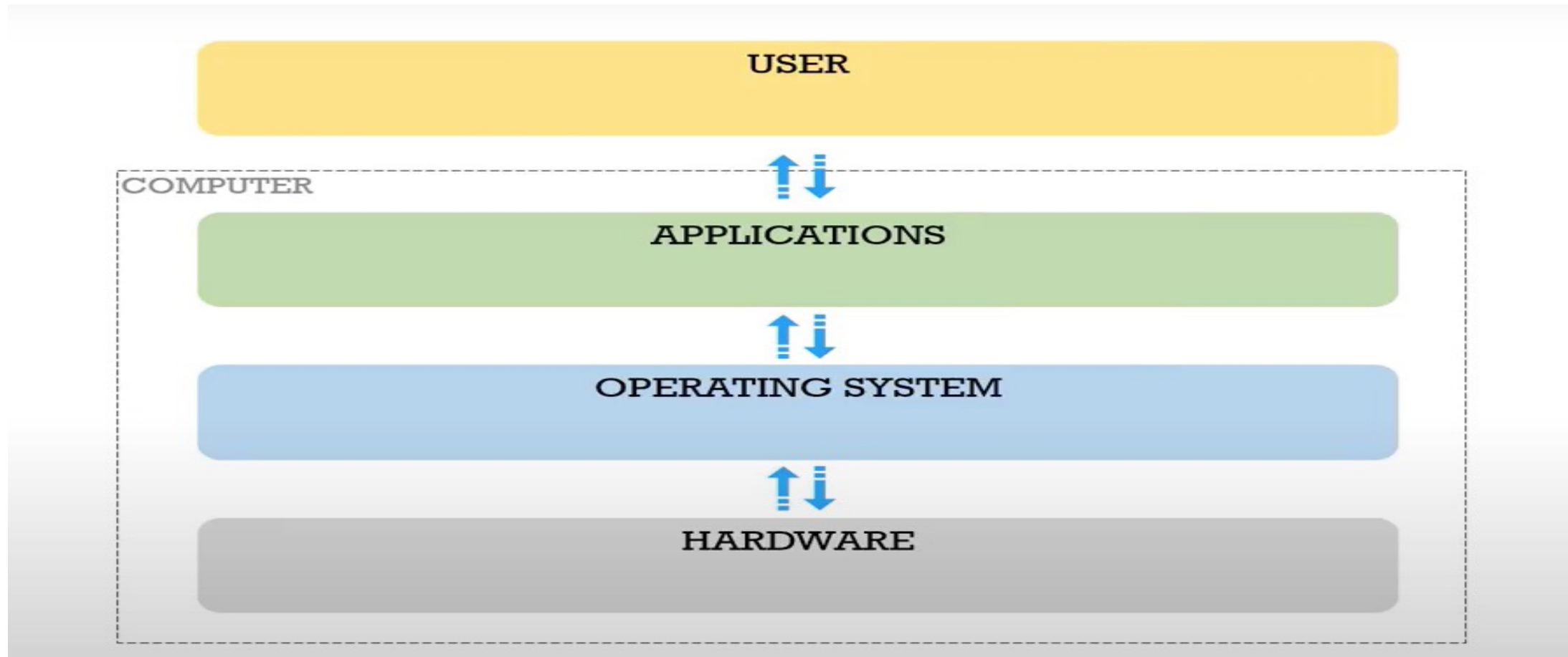


What is an Operating System?

- The OS is a program
 - Permit easy access, control hardware, communicate between user and software, user and hardware, software and hardware, maintain the computer system (hardware and software)
 - Users can access the OS via GUI and command line
 - Popular desktop OS: Windows, Linux, Mac OS
 - Popular mobile OS: Google's Android and Apple's iOS
 - OS is divided into four categories: Kernel, Device drivers, shell and Utility programs

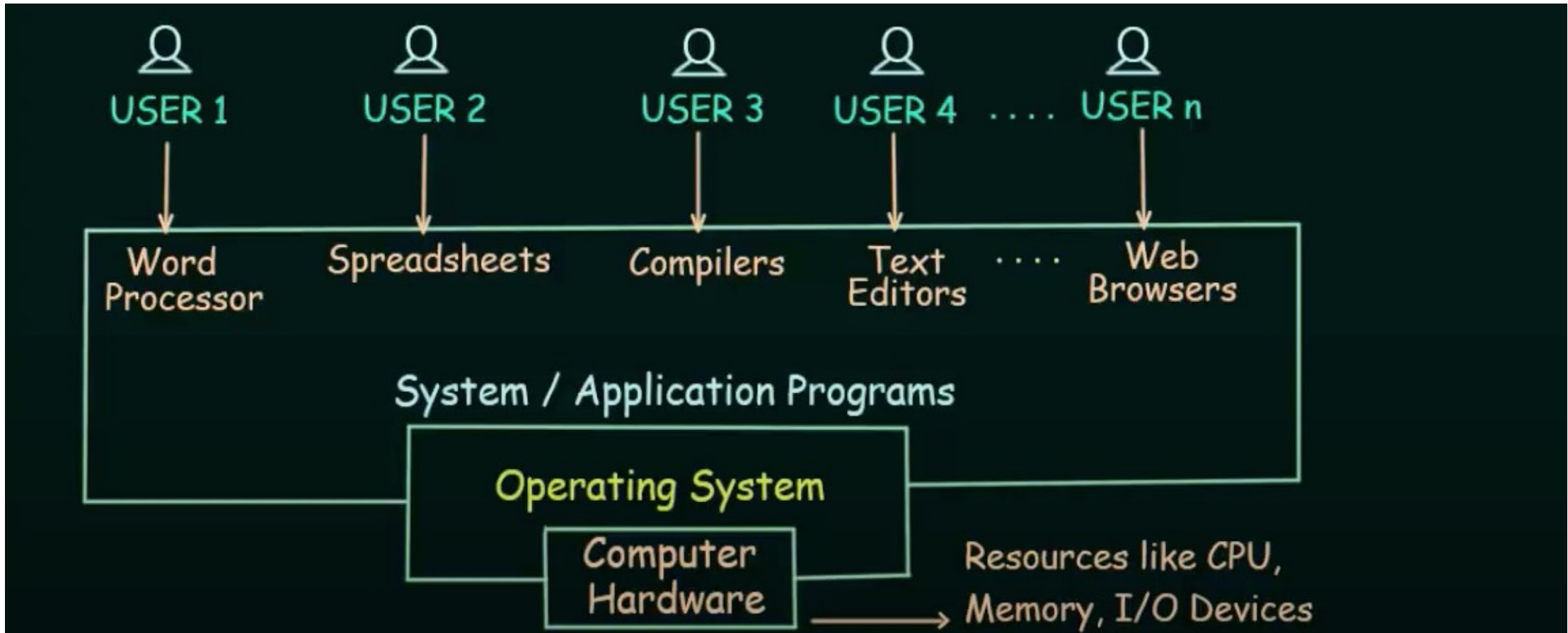


What is an Operating System..?





What is an Operating System..?





Common Operating Systems..



Windows



Linux



Ubuntu



Mac OS X
iOS



Android



Operating System Continued...

- Kernel : The software that contains the core components of the operating system
 - Always resident in memory
 - Responsible for primary OS tasks (process management, memory management, resource management, protection, security)
- Device drivers
 - Program interfaces between OS and peripheral devices e.g. printer drivers
- Shell
 - User interface, tailorable
- Utility programs
 - Add on programs to further manage and fine-tune your system (anti-viral programs, disk utilities, screen savers, etc)



Interrupts

- The CPU's fetch-execute cycle runs continuously unless interrupted
 - Interruptions can come from hardware or the running program
 - control+alt+delete
 - mouse moved
 - printer out of paper
 - program has run-time error
 - An interrupt interrupts the CPU at the end of fetch-execute cycle
 - Upon interrupt, CPU determines what device (or user or software) raised interrupt
 - select the proper interrupt handler (piece of OS code)
 - execute interrupt handler to handle the interrupt
 - resume the interrupted process

TABLE 4.1 Types of Interrupts

	Reason(s) for Interrupt
Disk drive	File not found Disk not in drive Disk not formatted
Keyboard	User enters keystroke User presses ctrl+alt+del
Mouse	Mouse moved Mouse button pressed or depressed
Network/MODEM	Message arrives
Printer	Paper jam Printer out of paper Printout complete
Program	Run time error (e.g., divide by 0, bad user input) Requires communication with another program
Timer	Timer elapses



OS Tasks

The OS is basically a computer system manager. It is in charge of the:

- user interface
- process and resource management
- memory management
- file management
- protection
- and security.



User Interface

- GUI
 - Point and click, drag, double click
 - Touch screen devices use gesture-based motions (swiping, tapping, pinching, reverse pinching)
- Command line
 - Enter commands from a command line prompt
 - Commands executed by an interpreter
 - breaks instructions into component parts
 - converts instructions to machine code and executes them
 - maintains a “session”
 - Commands may look cryptic and be hard to learn but offer more power and flexibility
 - Linux command: `find ~ -name 'core*' -exec rm {} \;`



Process Management and Scheduling

- Process – a running program
 - Processes have a status (running, ready, waiting, stopped)
 - And data (stored in memory, cache, registers)
- Process management is how the OS handles the tasks of starting processes, managing running processes, performing interprocess communication, terminating processes
- Another aspect is process scheduling – selecting the next process to run
- Scheduling algorithms include round robin, priority, first come first serve, shortest job first, longest job first
- Processes wait in queues
 - Waiting queue – waiting to be loaded into memory
 - Ready queue – in memory, but not currently executing by the CPU
 - I/O queue – waiting for I/O to complete

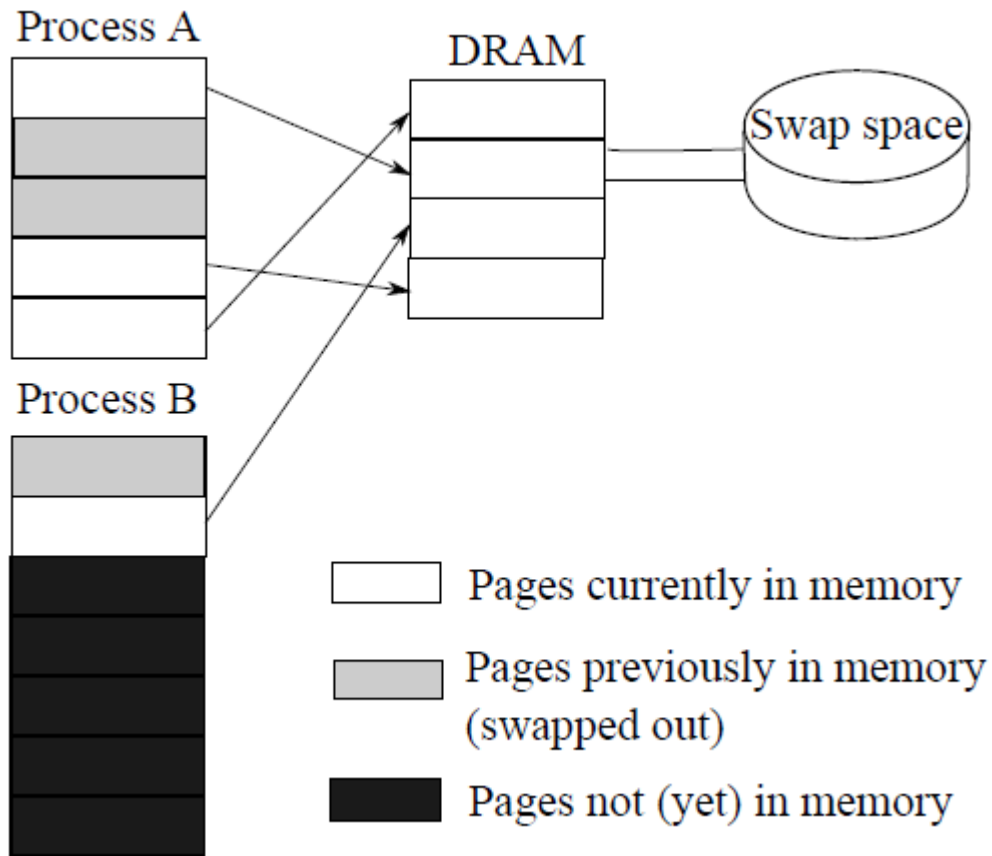


Memory Management

- We may not be able to fit all of our running program(s) in memory
 - We use swap space as “backing storage”
 - Swap space is placed on the hard disk
- We break our programs into fixed sized units called pages
 - The OS moves pages for us between swap space and memory as needed (on demand)
- This is known as virtual memory
- Program broken into fixed sized pages
 - Memory broken into fixed sized frames
 - when process begins running, first pages are moved to available frames
 - OS maintains page table
 - CPU generates a memory address:
 - page #, position on page
 - Use page table to translate this to physical address
 - frame #, position on frame
 - If page not in memory, page fault occurs



Virtual Memory: Paging and Page Tables



Process A page table

Page	Frame	Valid
0	1	T
1	-	F
2	-	F
3	3	T
4	0	T

Process B page table

Page	Frame	Valid
0	-	F
1	2	T
2	-	F
3	-	F
4	-	F
5	-	F
6	-	F



Page Faults and Swapping

- A page fault causes an interrupt
- OS uses a replacement strategy to find frame to free
 - if frame is of a page that has been modified, the page has to be saved back to swap space
 - once a frame is free, new page is loaded from swap space into memory
 - page table updated
- Movement of pages from swap space to memory (and memory to swap space) is swapping
 - swapping slows down the process because disk access time is much slower than memory access time

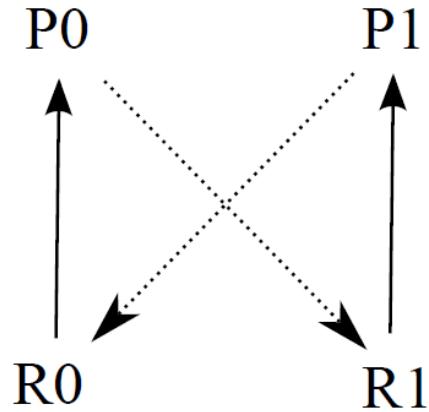


Resource Management

- The OS is also responsible for maintaining all other system resources
 - disk files, tape drives, flash drives
 - network access
 - printer access
- Most resources require mutually exclusive access
 - no more than 1 process can access a device at a time, others must wait until device is free
- Multitasking processes P0 & P1
 - P0 opens access to file F0
 - P0 reads datum X
 - P0 adds 10 to X
 - CPU switches to P1
 - P1 opens access to file F0
 - P1 reads datum X
 - P1 subtracts 300 from X
 - P1 writes new value to F0
 - CPU switches to P0
 - P0's value of X is now incorrect
- This is data corruption



Deadlock



Solid lines indicate granted resources
Dotted lines indicate requested resources

- The result is that neither P0 nor P1 can continue
 - We would have to kill one of the processes to let the other continue (restart the killed process later)
 - OSs will often ignore deadlock and require that the user discover it and kill one of the processes off
-
- In the above scenario, P0 is using R0, P1 is using R1, P0 wants to access R1 and P1 wants to access R0
 - Deadlock



Other OS Tasks

- File management
 - Users dictate file system operations through windows explorer or command line instructions (DOS, Linux)
 - creating directories, moving files, copying files, deleting files, etc
 - Every file operation is a request
 - the OS must ensure that the user has access to the given resource to perform the operation
- Protection
 - Resources need protection from misuse
 - Each resource has permissions associated with it (access, read, write, execute)
 - User accounts/authentication are used to establish protection
- Security extends permissions across a network
 - A user can then remotely control the computer



Forms of Process Management

- Can the OS run more than one program at any time?
 - not simultaneously but concurrently
- One process at a time
 - single tasking
 - batch processing
- Concurrent processing
 - multiprogramming
 - multitasking
 - multithreading
 - multiprocessing



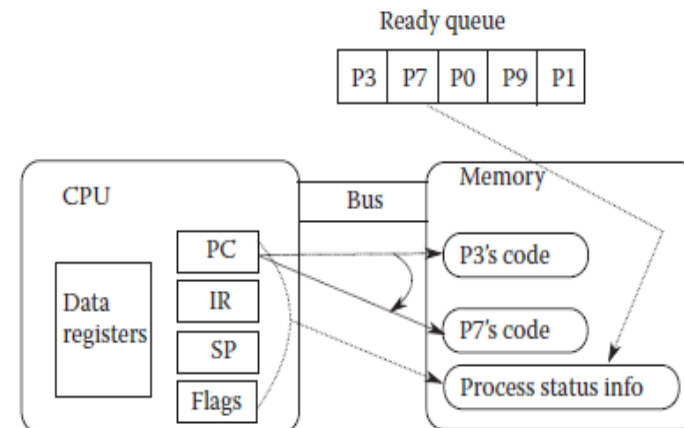
Single Tasking and Batch Processing

- Single Tasking
 - User starts the process
 - Process runs to completion
 - If I/O is needed, CPU waits
 - User is not allowed to run more than 1 program at a time
 - Most early operating systems were single tasking
 - Most PC operating systems were single tasking until mid 90s
- Batch Processing
 - For multiple user systems
 - users submit processes at any time
 - off-line system receives requests
 - OS schedules processes
 - Otherwise, very similar to single tasking (one program at a time)
 - No interactivity because the process may not be executed until the user is gone
 - input supplied with the program (e.g., punch cards, magnetic tape)
 - output sent to off-line source (tape, printer)



A Context Switch

- Concurrent processing needs a mechanism for the CPU to switch from one process to another
 - New process needs to be loaded into memory
 - Old process' status (PC, IR, stack pointer, status flags, etc) saved to memory
 - New process status (register values) restored (from memory)
 - During the switch, the processor is idle
- Ready queue – stores those processes available for switching
 - These are processes loaded into memory which have either started execution and been paused or can start execution
- Below, we see a switch between process P3 and P7





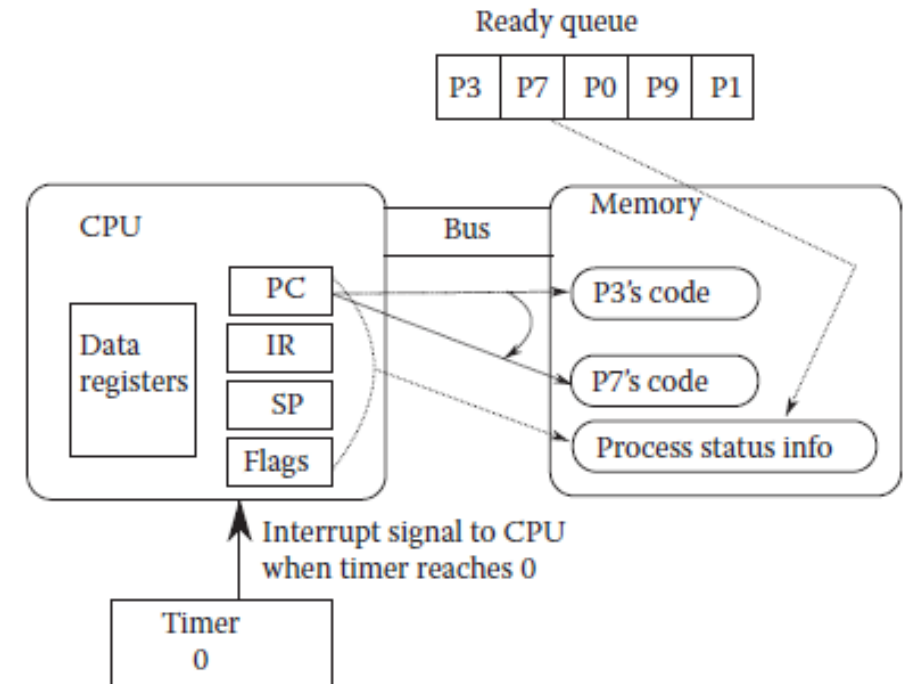
Multiprogramming & Multitasking

- Multiprogramming (or cooperative multitasking) is like batch processing in that one process is executed at a time except
 - When a process requires I/O, it is moved to an I/O queue
 - Context switch to next waiting process
 - When process finishes with I/O, switch back to it
 - More efficient than batch processing or single tasking because the CPU does not remain idle during time consuming I/O, only delay is during the context switches
- For multitasking, add a timer and before a process starts, set the timer to some value (e.g., 10,000)
 - After each machine cycle, decrement the timer
 - When timer reaches 0, force a context switch to the next process in the ready queue
 - User will not notice the time it takes to cycle through the processes



Multitasking

- More appropriately called competitive (or pre-emptive) multitasking
- Computer appears to be executing two or more processes simultaneously
 - it is switching quickly between processes
- Most OSs do both cooperative and competitive multitasking today



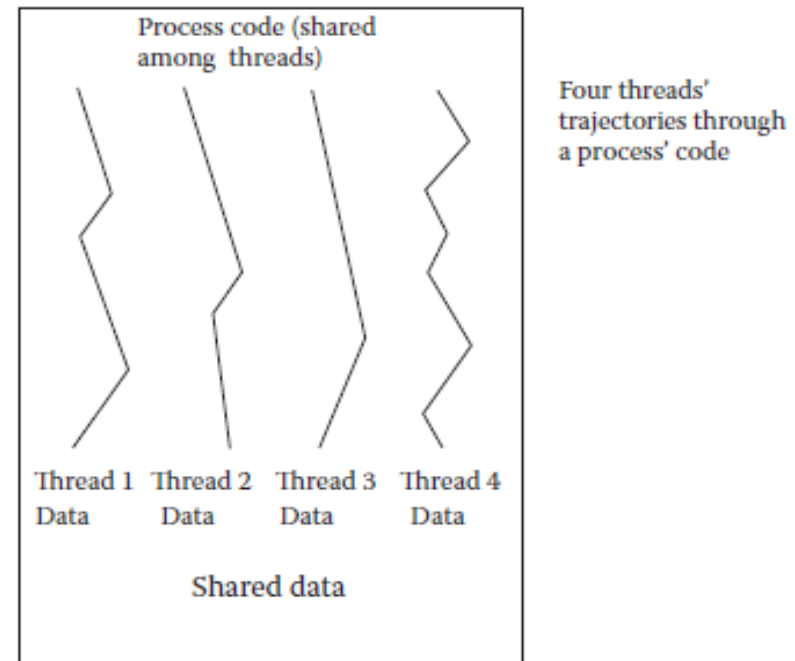
Competitive multitasking was originally called time sharing in the 1960s



Threads & Multithreading

- Threads – multiple instances of the same process sharing the same code
 - But with separate data
- For instance, you might have 3 Firefox windows open, these are threads of the same process
- Threads make their way through the same code along different paths
 - See figure to the right

- Multithreading is multitasking across both processes and threads
 - Switching between threads is simpler (less time consuming)





Multiprocessing

- Many computers today have multiple processors
 - Or multiple cores on one chip (a core is basically a processor which shares pins and a cache with other cores on the same chip)
- Multiprocessing is multitasking spread across multiple processors
- Most OSs are capable of multiprocessing but do not necessarily share the cores effectively
 - For instance, if you have 4 cores, you would not achieve a 4 times speedup over a computer with a single core processor



Booting

- Main memory (DRAM, SRAM) is volatile
 - Turn off the power, lose the contents
 - Turn on the power, memory is empty
- We need the OS loaded and running in memory to load and run programs
- If you turn your computer on, how do we get the OS loaded into memory and running when the OS is needed to load programs into memory to run them?
 - We need a 1-time process – booting
 - The boot process (or portions of it) are stored in ROM (non-volatile memory)



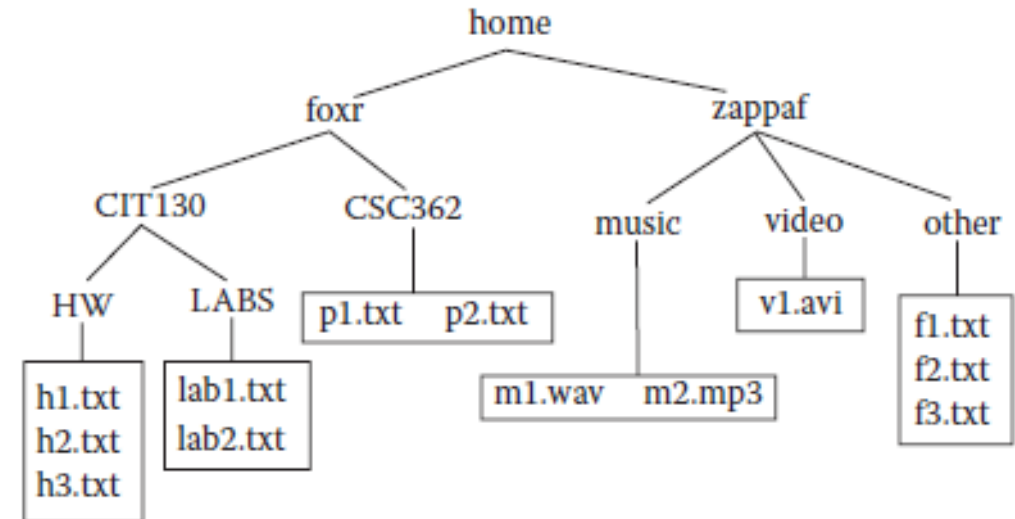
The Boot & Initialization Process

- CPU initializes itself (registers, control signals)
- BIOS performs power on self test
- Disk controllers tested
- BIOS determines where the OS is stored
- Boot loader runs to load OS (if multiple OSs available, user might be able to select)
- OS kernel loaded and initialized
- OS runs initialization scripts
 - Establish runlevel (Linux is a number from 0 to 6, windows is safe mode, safe mode with network, user mode, etc)
- OS running, computer ready for user
 - BIOS is stored in ROM
 - Kernel often stored on hard disk (possibly also available on optical disk, flash drive or over network)



Moving Around the Linux File System

Command/Result	Explanation
<code>\$ pwd</code> <code>/home/foxr</code>	print working directory
<code>\$ cd CIT130</code> <code>\$ ls</code> <code>HW LABS</code>	change to the CIT130 subdirectory list the contents
<code>\$ ls -l</code> <code>drwxr-xr-x 2 foxr foxr</code> <code>1024 Jan 20 03:41 HW</code> <code>drwxr-xr-x 4 foxr foxr</code> <code>4096 Jan 21 17:22 LABS</code>	perform a long listing of the contents





Continued

\$ cd HW	change to the HW directory
\$ cp h1.txt /home/foxr/ CIT130/LABS	copy the h1.txt file to the LABS directory
\$ cd ..	move up one level (/home/foxr/ CIT130)
\$ mv LABS/lab1.txt .	move lab1.txt to here
\$ ls ../../CSC362 p1.txt p2.txt	list the contents of the CSC362 directory
\$ cd ~/CSC362	change to your CSC362 directory
\$ rm *.*	delete everything here

rm: remove regular file 'p1.txt'? y

rm: remove regular file 'p2.txt'? y

rm: remove regular file 'lab1.txt'? n



Continued

\$ cd ..	move up to foxr
\$ rmdir CSC362	delete the entire CSC362 directory
rmdir: CSC362: Directory not empty	

\$ cd /	move to the top of the file system
\$ cd home	move down to the home directory
\$ ls	list the contents
foxr zappaf	
\$ cd zappaf	move to zappaf's directory
\$ cp music/*.* ~	copy files in music to your directory

\$ mv videos/*.* ~	move everything
mv: cannot move `/home/ zappaf/videos/v1.avi' to `./v1.avi': Permission denied	



Continued

<code>\$ cd ~</code>	return to your home directory
<code>\$ mkdir STUFF</code>	create a directory called STUFF
<code>\$ ln -s videos/v1.avi STUFF/video</code>	create a symbolic link

<code>\$ ls STUFF</code>	list contents of the STUFF directory
<code>video</code>	

```
$ ls -l STUFF  
  
lrwxrwxrwx 1 foxx foxx 16 Jan 21 17:13  video -> /home/zappaf/videos/v1.avi
```



DOS

Command/Result	Explanation
C:	switch to C: partition/drive
C:\> cd Users\foxr	move to foxr directory
C:\Users\foxr> dir	list the contents
Wed 11/23/2011 10:53 AM <DIR>	.
Wed 11/23/2011 10:53 AM <DIR>	..
Wed 11/23/2011 10:55 AM <DIR>	Desktop
Wed 11/23/2011 10:55 AM <DIR>	Documents
Wed 11/23/2011 10:55 AM <DIR>	Downloads
...	
0 File(s)	
15 Dir(s)	



Continued

C:\Users\foxr> cd Documents	change to Documents directory
C:\Users\foxr\Documents> copy *.* \Users\zappaf	copy everything to zappaf's directory
C:\Users\foxr\Documents> cd ..	move up 1 level
C:\Users\foxr> cd Downloads	move down to Downloads
C:\Users\foxr\Downloads> move *.* \Users\zappaf	move instead of copy
C:\Users\foxr> cd \ Users\zappaf	switch to zappaf's directory

C:\Users\zappaf> dir

C:\Users\zappaf> del *.*

C:\Users\zappaf>